Algoritmos Distribuidos
Transferencia de datos en ambiente no fiable

Andrés Sanoja
Claudia León

Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

Octubre 2008 – Marzo 2022



Objetivo

- Estudiar soluciones al problema de transferencia de información entre dos procesos en un ambiente no fiable.

Transferencia de datos

- Una entidad llamada Productor envía mensajes a otra llamada Consumidor
- Los mensajes producidos y aún no consumidos deben ser memorizados hasta su consumo
- Una vez emitido, un mensaje debe ser consumido una sóla vez
- Suponemos una capacidad de almacenamiento de n mensajes

Especificación de las restricciones

Cuando una llamada a la operación *Oper* es autorizada, se incrementa automaticamente el contador: #aut(Oper)

Cuando una llamada a la operación *Oper* finaliza, se incrementa automaticamente el contador: #term(Oper)

Caso: ambientes no fiables

Nuevas Hipótesis:

- Pérdida de mensajes,
- Duplicación de mensajes,
- Mensajes alterados,
- Mensajes fuera de orden.

Solución: Construir un algoritmo o protocolo distribuido que preste el mismo servicio, que el ambiente fiable esto es:

Si en un momento dado la secuencia de mensajes producidos es:

$m_1\ m_2\ m_3\ .\ .\ .\ .\ .\ .\ .\ m_i\ m_{i+1}\ .\ .\ .\ .\ .\ m_j\ .\ .\ .\ .\ mk$

entonces la secuencia de mensajes consumidos debe ser:

$m_1\ m_2\ m_3\ .\ .\ .\ .\ .\ .\ .\ m_i\ m_{i+1}\ .\ .\ .\ .\ .\ m_j$

con: $k - n \leq j \leq k$ (existe a lo sumo n mensajes producidos y aún no consumidos)

Detección de alteración de mensajes

- Suponemos la existencia de un mecanismo subyacente de detección de mensajes alterados, que produce la pérdida de dichos mensajes
- Si se resuelve el problema de pérdida y se sabe detectar los mensajes alterados, entonces se tiene resuelto el problema de alteración de mensajes

Fundamentos de la solución

- El productor va a utilizar un contador monótono creciente, el cual le permitirá asociar un número de secuencia a cada mensaje enviado
- Los mensajes tendrán el formato:

  (número de secuencia, valor del mensaje)
- La duplicación se detecta fácilmente: dos mensajes con el mismo número de secuencia, son duplicados
- Similarmente son detectados los mensajes fuera de secuencia: el número de secuencia debe presentarse en orden creciente
- Como reconocer la pérdida de un mensaje? Un hueco en el número de secuencia puede significar que el mensaje se perdió ó que fue “pasado” por otro de número mayor.

Detección de pérdida de mensajes

- El consumidor va a emitir un mensaje de acuso de recibo por cada mensaje recibido
- Si al cabo de cierto tiempo (time-out) el mensaje no ha sido confirmado por el receptor, se re-envía el mensaje

Problemas: Calibración del time-out

- Un valor muy pequeño produce re-emisiones incesantes, numerosas duplicaciones y congestionamiento de las líneas
- Un valor muy grande implica que un mensaje perdido será re-transmitido después de largo tiempo

Detección de pérdida de mensajes

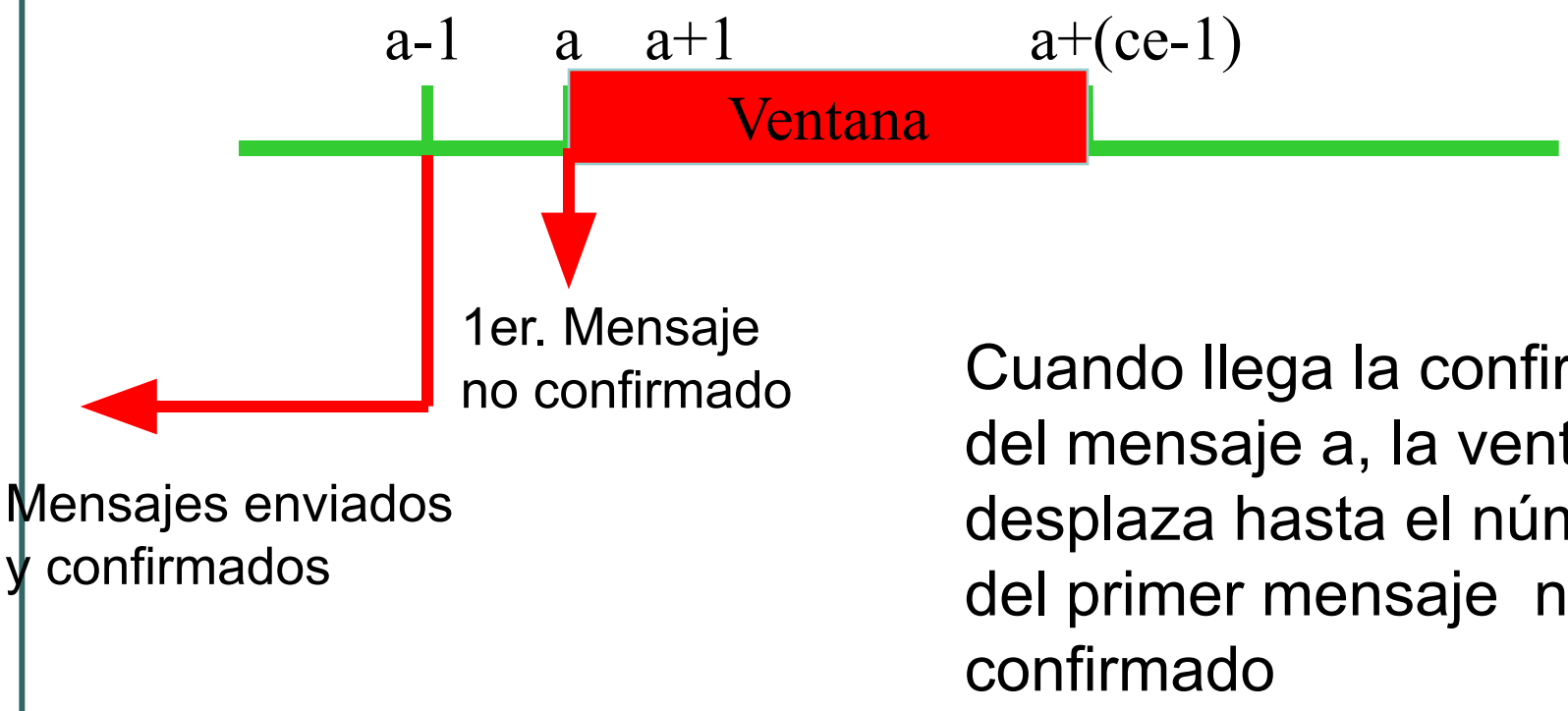
Problema: Utilización de las líneas

Si sólo se acepta el mensaje esperado y se espera el acuso de recepción para emitir el mensaje siguiente, entonces las líneas son sub-utilizadas

Solución: Créditos o ventana

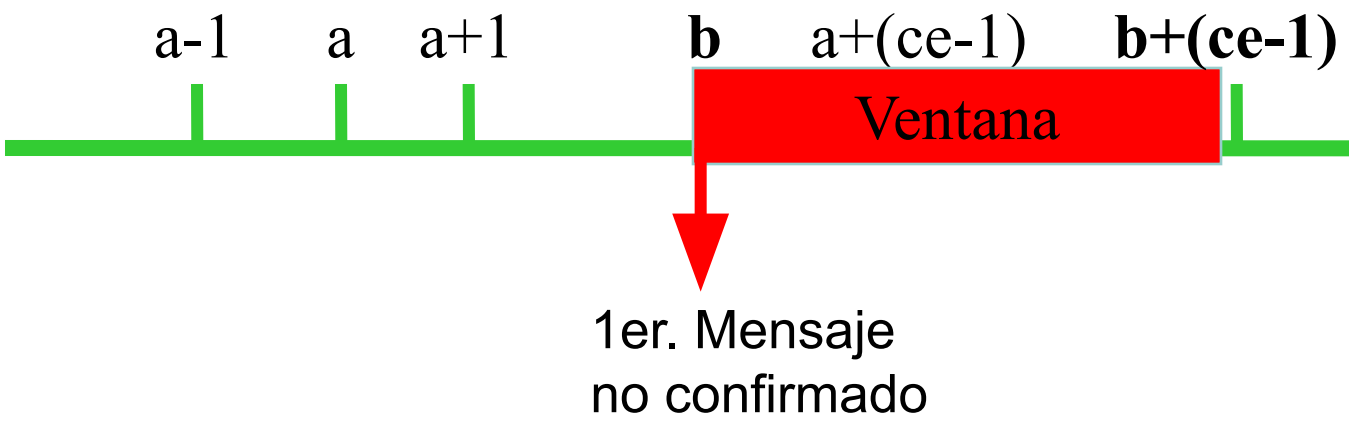
Crédito(CE): Cantidad de mensajes consecutivos que el emisor puede enviar sin haber recibido confirmación

Ventanas y créditos **Productor**



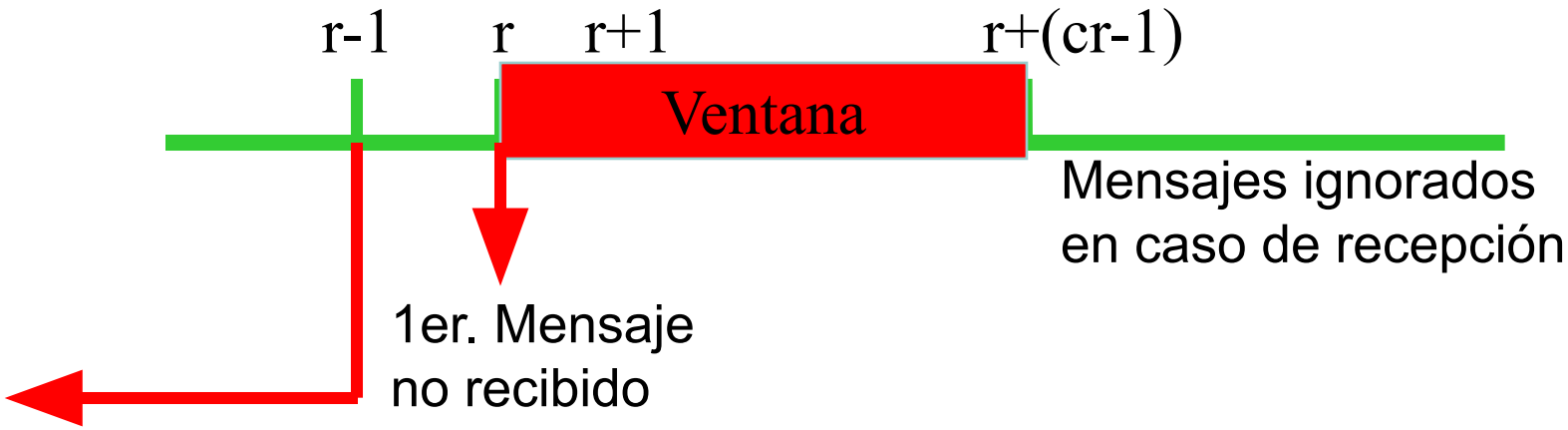
Cuando llega la confirmación del mensaje a, la ventana se desplaza hasta el número b del primer mensaje no confirmado

Ventanas y créditos



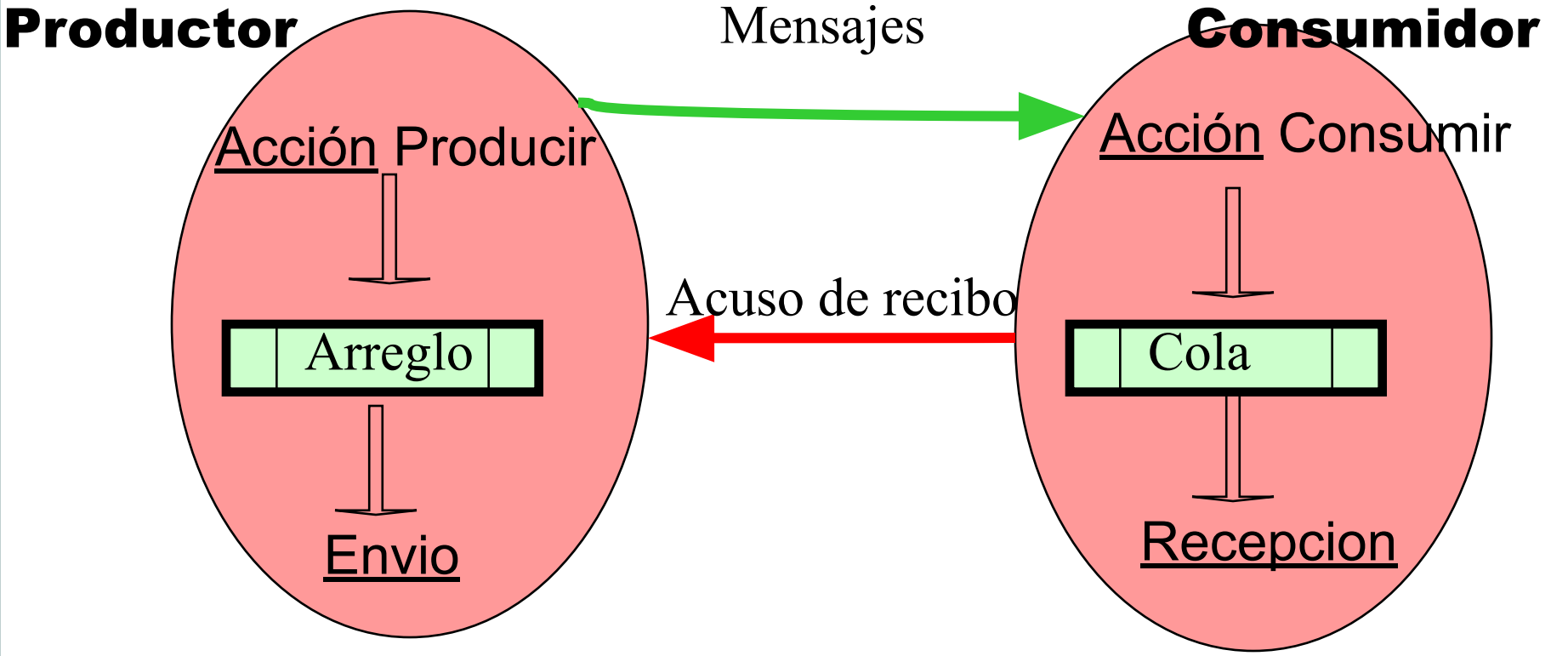
Ventanas y créditos

Consumidor



Cuando el mensaje r es recibido, el receptor lo confirma y hace desplazar la ventana hasta [s,s+(cr-1)] donde s designa el número del primer mensaje aún no recibido

Una Solución



Condición : autPro - termCon < n

ce ≤ n, cr = 1

Nodo del Productor

```
Var NSecSig:entero inicializa 0; %Próximo Nro.de secuencia a producir
    NMenEnv:entero inicializa 0; %Próximo nro. de secuencia a enviar
    Nacuso: entero inicializa 0;%Nro. más pequeño confirmación esperada
    Guardar: arreglo [0..n-1] de mensaje,
    Lleno : 0..n inicializa 0 %De posiciones ocupadas en el arreglo
                %guardar. Equivalente a la condición
%autPro - CtermCon < n

Cuando  { reciba mensaje tipo Acuso(x:Entero) } hacer
    Si  x ≥ Nacuso entonces
        Lleno := Lleno - (x+1 - Nacuso); Nacuso:= x +1;
    fsi;
fuando;
Cuando  { se quiera retransmitir}  hacer
    NMenEnv:=Nacuso;
fcuando
```

Nodo del Productor (continuación ...)

```
Var NSecSig:entero inicializa 0;%Prñoximo nro. de secuencia a producir
    NMenEnv:entero inicializa 0;%Próximo nro. de secuencia a enviar
    Nacuso:entero inicializa 0; %Nro. más pequeño confirmacion esp.
    Guardar: arreglo [0..n-1] de mensaje,
    Lleno : 0..n inicializa 0 %Nro.posiciones ocupadas en el areglo

Cuando  {se invoque a Producir(m: mensaje)} hacer
Si  Lleno < n entonces
    Lleno := Lleno + 1; Guardar[NSecSig mod n] := m;
    NSecSig:=NSecSig+1;
fsi;
fcuando;
Cuando  {se quiera Enviar } hacer
Si (NMenEnv < NSecSig and NMensEnv ≤ Nacuso + (ce-1) ) Entonces
    Enviar (NMenEnv, Guardar[NMenEnv mod n] ) a Consumidor;
    NMenEnv:=NMenEnv +1;
fsi;
fcuando
```

Nodo del Consumidor

```
Var C: Cola  de mensaje inicializa vacia;
       Mcon: o..n inicializa 0; %mensajes disponibles para consumir
       Nespera : entero inicializa 0; %mensaje esperado
       Nacuso: entero inicializa -1; %Ultimo acuso enviado

Cuando  se invoque a Consumir (m: mensaje) hacer
    Si  Mcon > 0 entonces
        Mcon := Mcon - 1;  (Nacuso,m) := Eliminar(C);
        Enviar (acuso,Nacuso) a Productor;
    fsi; fcuando;
Cuando  llegue  mensaje (NroSec:entero,m: mensaje)  hacer
    Si NroSec = Nespera entonces
         Mcon:= Mcon +1; Insertar((NroSec,m),C); Nespera:=Nespera + 1;
    fsi; fcuando
Cuando  {quiera retransmitir}  hacer
    Enviar(acuso,Nacuso) a Productor;
fcuando;
```

